



BSc (Hons) in **Information Technology**
Specialized in AI
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent`.
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

Table-Driven Agent is a kind of agent that has an action defined for every possible sequence of percept inputs it might encounter. The approach described above cannot be applied to the complex environment such as Chess, as there are numerous possible board states and percept sequences. In order to construct an absolutely mathematically correct Table-Driven Agent, we would have to store the action that should be performed in every single possible state.

As the agent lives longer, it continues receiving the information from the environment in the form of percept input. As it receives new information from the environment, it adds this information to the current percept sequence, thus, increasing the size of this sequence. The amount of possible percept sequences will grow exponentially in

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

In the SimpleReflexAgent I developed, the Condition-Action Rules are implemented in the method `sense_and_act(self, percept)`. The decision making of the agent is done based on the checking the current percept and immediately choosing the appropriate action via IF-THEN rules.

The agent does not store any information regarding the previously performed actions or the state of the environment.

Firstly, the agent checks whether there is any food in the current cell. In case when the value of the parameter `percept["food_here"]` equals `True`, the "Suck" action is immediately performed in order to suck the food. In case when there is no food in the cell, the agent proceeds to check whether there is a wall in front of it or not. In case of `percept["wall_ahead"]` being `True`, the "TurnLeft" action is chosen. Otherwise, in case none of the above conditions holds true, the "Forward" action is performed. 

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

SimpleReflexAgent went into an endless loop since it operated in a partially observable environment with no memory. It could only perceive its current perception like seeing a wall in front of it or food in the current location. It could not recall that it had been at a particular place before.

This is why when the agent perceived the same percept, it performed the same 

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In Step 1.3, the agent I have created employs the internal state of the model-based agent to store information about the environment. The Transition Model is accomplished by maintaining the internal state of the agent, which includes its current location, cells that the agent has already visited, and direction the agent is facing when performing an action.

The Sensor Model is achieved through using the current percept (`wall_ahead` and `food_here`) to update the agent's understanding of the environment. In particular, 

Finalize and Lock Lab 02 Documentation

